

Critical Thinking Module 2 – Program Explanation and Screenshots

For this assignment, I selected a puppy image.

Program Explanation

The program uses OpenCV to execute color channel transformations on the puppy image. It first loads the image as a (257, 250, 3) NumPy array, which means 257 rows, 250 columns, and 3 color values per pixel in blue, green, and red (BGR) order. Here, the array's third dimension stores the color channels, not spatial depth. The values are stored as `uint8`, that is, unsigned 8-bit integers ranging from 0 to 255; for example, a black BGR pixel is `[0, 0, 0]` and a white BGR pixel is `[255, 255, 255]`. The NumPy image array is indexed by row first, so `image[y, x]` accesses the pixel at row `y` and column `x`. Using `cv2.split()`, the program separates the image into three (257, 250) matrices, one for each color channel, blue, red, and green. Then it uses grayscale to represent each color intensity: the darker the pixels, the lower the value of the color is, while the brighter the color is, the higher its value is. The program then uses `cv2.merge((blue, green, red))` to put the original image back together. As a final step, the program swaps the red and green using `cv2.merge((blue, red, green))`. The original red values become the output's green values, and the original green values become its red values. The output still uses BGR storage; in RGB number values, 0 to 255. I noticed that the puppy's fur became green and the water pink, while each pixel stays in the same row and column. In other words, the channel swaps the red and green color values within each pixel the position of each pixel. The code performs these operations through `cv2.split()` and `cv2.merge()`. The program terminal outputs provide explanations with matrices. It also saves the modified images as PNG files. Finally, the Program uses `iTerm2Image` protocol to display the PNG image in the terminal; to run properly, the program needs to be executed in a terminal capable of supporting the `iTerm2Image` protocol.

Color channel transformation matrices

These matrices describe the five operations shown in the program. They act on one pixel's color values, not its row and column. The program uses `cv2.split()` and `cv2.merge()` to perform the operations; it does not explicitly multiply each pixel by these matrices.

$$C = \begin{bmatrix} B \\ G \\ R \end{bmatrix}, \quad C' = \begin{bmatrix} B' \\ G' \\ R' \end{bmatrix}$$

Here, B, G, and R are the original channel values at one pixel. The prime mark (') identifies an output value, not a derivative. The T in the paragraph's vector notation means transpose, so the values are written as a column for matrix multiplication.

Extracting the three channels

Each 1 selects a channel, and the two 0s leave out the other channels. Applying the selection at every pixel produces one 2D matrix per channel.

$$\begin{aligned} R &= [0 \quad 0 \quad 1] C = 0B + 0G + 1R \\ G &= [0 \quad 1 \quad 0] C = 0B + 1G + 0R \\ B &= [1 \quad 0 \quad 0] C = 1B + 0G + 0R \end{aligned}$$

Reconstructing the original image

`cv2.merge((blue, green, red))` returns each channel to its original position. Splitting and merging in this order is equivalent to applying the identity matrix:

$$C' = \begin{bmatrix} B' \\ G' \\ R' \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} B \\ G \\ R \end{bmatrix} = \begin{bmatrix} B \\ G \\ R \end{bmatrix}$$

Exchanging red and green

`cv2.merge((blue, red, green))` exchanges the green and red positions. This permutation matrix rearranges the channel values while keeping blue unchanged:

$$C' = \begin{bmatrix} B' \\ G' \\ R' \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} B \\ G \\ R \end{bmatrix} = \begin{bmatrix} B \\ R \\ G \end{bmatrix}$$

Pixel example from the screenshot

The center pixel is `image[128, 125] = [82, 51, 78]` in BGR order. The swap produces:

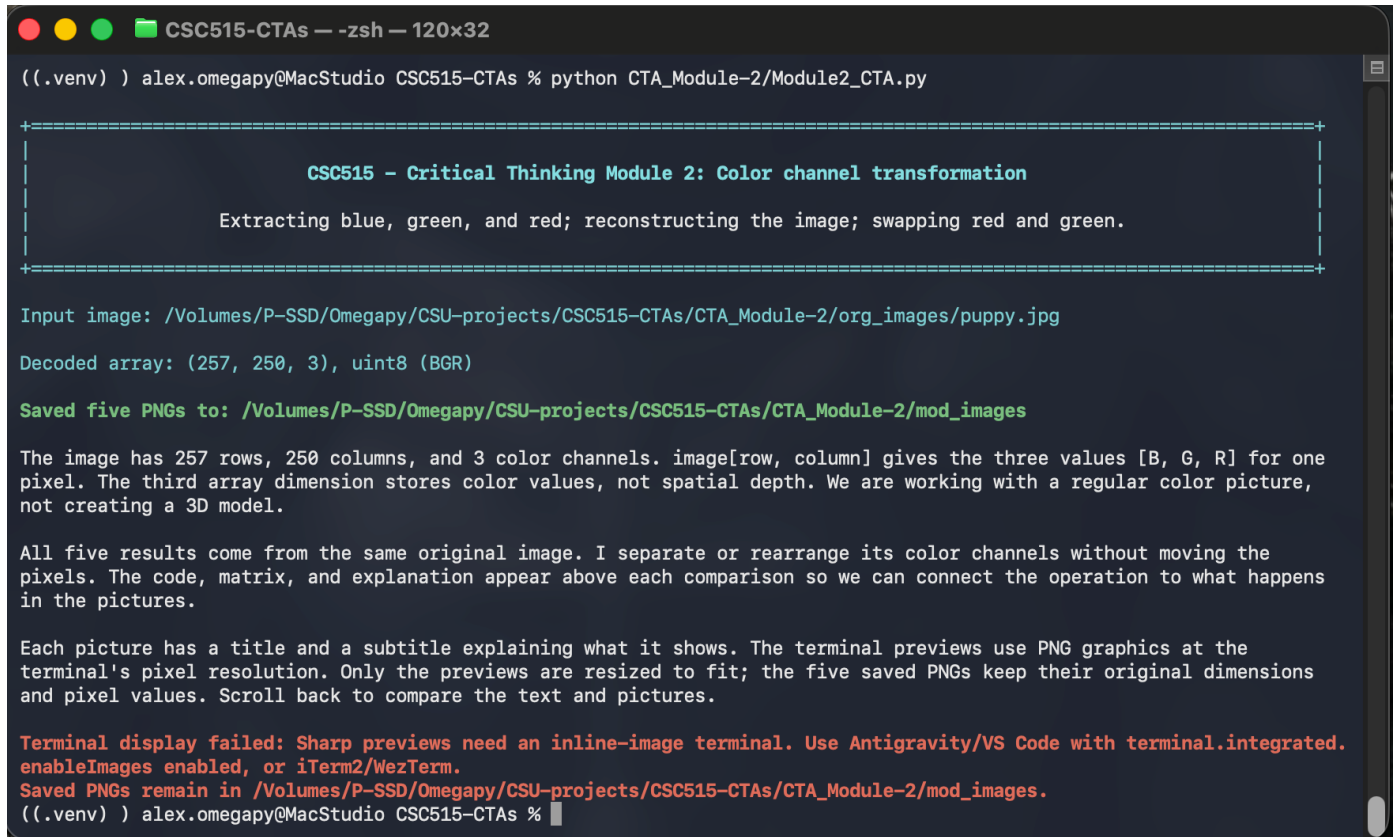
$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 82 \\ 51 \\ 78 \end{bmatrix} = \begin{bmatrix} 82 \\ 78 \\ 51 \end{bmatrix}$$

Please see the `README.md` file for more information

Screenshots

The following image is a Terminal display failure.

Trying to use a terminal that does not support the Term2Image protocol will generate an error.



```
CSC515-CTAs — -zsh — 120x32

((.venv) ) alex.omegapy@MacStudio CSC515-CTAs % python CTA_Module-2/Module2_CTA.py

=====
|                                     |
|      CSC515 - Critical Thinking Module 2: Color channel transformation      |
|                                     |
|      Extracting blue, green, and red; reconstructing the image; swapping red and green.      |
|                                     |
|=====|

Input image: /Volumes/P-SSD/Omegapy/CSU-projects/CSC515-CTAs/CTA_Module-2/org_images/puppy.jpg
Decoded array: (257, 250, 3), uint8 (BGR)

Saved five PNGs to: /Volumes/P-SSD/Omegapy/CSU-projects/CSC515-CTAs/CTA_Module-2/mod_images

The image has 257 rows, 250 columns, and 3 color channels. image[row, column] gives the three values [B, G, R] for one pixel. The third array dimension stores color values, not spatial depth. We are working with a regular color picture, not creating a 3D model.

All five results come from the same original image. I separate or rearrange its color channels without moving the pixels. The code, matrix, and explanation appear above each comparison so we can connect the operation to what happens in the pictures.

Each picture has a title and a subtitle explaining what it shows. The terminal previews use PNG graphics at the terminal's pixel resolution. Only the previews are resized to fit; the five saved PNGs keep their original dimensions and pixel values. Scroll back to compare the text and pictures.

Terminal display failed: Sharp previews need an inline-image terminal. Use Antigravity/VS Code with terminal.integrated.enableImages enabled, or iTerm2/WezTerm.
Saved PNGs remain in /Volumes/P-SSD/Omegapy/CSU-projects/CSC515-CTAs/CTA_Module-2/mod_images.
((.venv) ) alex.omegapy@MacStudio CSC515-CTAs %
```

See next page

The following images are from the Antigravity IDE terminal, which support Term2Image protocol

STEP 1/5: Red channel intensity

```
CSC515-CTAs - README.md - Untracked
Problems Output Debug Console Terminal Ports
alex.omegapy@MacStudio CSC515-CTAs % .venv/bin/python CTA_Module-2/Module2_CTA.py

=====
CSC515 - Critical Thinking Module 2: Color channel transformation
Extracting blue, green, and red; reconstructing the image; swapping red and green.
=====

Input image: /Volumes/P-SSD/Omegapgy/CSU-projects/CSC515-CTAs/CTA_Module-2/org_images/puppy.jpg
Decoded array: (257, 250, 3), uint8 (BGR)

Saved five PNGs to: /Volumes/P-SSD/Omegapgy/CSU-projects/CSC515-CTAs/CTA_Module-2/mod_images

The image has 257 rows, 250 columns, and 3 color channels. image[row, column] gives the three values [B, G, R] for one pixel. The third array dimension stores color values, not spatial depth. We are working with a regular color picture, not creating a 3D model.

All five results come from the same original image. It separates or rearranges its color channels without moving the pixels. The code, matrix, and explanation appear above each comparison so we can connect the operation to what happens in the pictures.

Each picture has a title and a subtitle explaining what it shows. The terminal previews use PNG graphics at the terminal's pixel resolution. Only the previews are resized to fit; the five saved PNGs keep their original dimensions and pixel values. Scroll back to compare the text and pictures.

=====
STEP 1/5: Red channel intensity
The original red values, shown as a 2D grayscale image
=====

This image keeps the red value from each pixel of the original picture. The blue and green values are left out of this result.

Darker pixels have lower red values; brighter pixels have higher red values. Black represents 0, and white represents 255 in this grayscale display.

Code: blue, green, red = cv2.split(image)

This is the red array returned by cv2.split(). In the matrix below, the 1 selects R, and the two 0s leave out the other channels. Each pixel now has one value instead of three, so the result is a 2D matrix. The grayscale display shows how much of this channel was present: 0 is none, and 255 is the highest value in this 8-bit image.

Reading the three pictures from left to right:

Original color image | Selected channel in its own color | 2D grayscale result.

The middle picture is a visual aid. It keeps the selected channel and sets the other two to zero in a separate preview array. It is not an extra processing step, and it does not convert that preview to get the grayscale result. The result on the right comes directly from cv2.split().

The matrix for one pixel (@ means matrix multiplication):

R = [0 0 1] @ [B]
      [G]
      [R]

R = 0*B + 0*G + 1*R

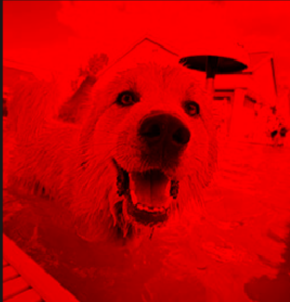
The matrix describes what happens to the color values. The code uses cv2.split() and cv2.merge() rather than multiplying each pixel by a matrix. The pixel's row and column do not change.

Example from the center pixel: image[128, 125] = [82, 51, 78] in BGR order.
Selected channel value: 0*82 + 0*51 + 1*78 = 78.

Array shape: (257, 250, 3) -> (257, 250). Data type: uint8.

Saved result: /Volumes/P-SSD/Omegapgy/CSU-projects/CSC515-CTAs/CTA_Module-2/mod_images/red_channel.png

Original color image | Red-only color view (visual aid) | Result: 2D red intensity
The same source for each comparison | Other two channels set to 0 | Original red values shown in grayscale
```



STEP 2/5: Green channel intensity

STEP 2/5: Green channel intensity

The original green values, shown as a 2D grayscale image

Here, we are looking at the green value from each original pixel. The result keeps those values without including blue or red.

A dark pixel has a low green value, and a bright pixel has a high green value. The display goes from black at 0 to white at 255.

Code: `blue, green, red = cv2.split(image)`

This is the green array returned by `cv2.split()`. In the matrix below, the 1 selects G, and the two 0s leave out the other channels. Each pixel now has one value instead of three, so the result is a 2D matrix. The grayscale display shows how much of this channel was present: 0 is none, and 255 is the highest value in this 8-bit image.

Reading the three pictures from left to right:

Original color image | Selected channel in its own color | 2D grayscale result.

The middle picture is a visual aid. It keeps the selected channel and sets the other two to zero in a separate preview array. It is not an extra processing step, and it does not convert that preview to get the grayscale result. The result on the right comes directly from `cv2.split()`.

The matrix for one pixel (@ means matrix multiplication):

$$G = \begin{bmatrix} 0 & 1 & 0 \end{bmatrix} @ \begin{bmatrix} B \\ G \\ R \end{bmatrix}$$

$$G = 0*B + 1*G + 0*R$$

The matrix describes what happens to the color values. The code uses `cv2.split()` and `cv2.merge()` rather than multiplying each pixel by a matrix. The pixel's row and column do not change.

Example from the center pixel: `image[128, 125] = [82, 51, 78]` in BGR order.

Selected channel value: `0*82 + 1*51 + 0*78 = 51`.

Array shape: (257, 250, 3) -> (257, 250). Data type: `uint8`.

Saved result: `/Volumes/P-SSD/0megapy/CSU-projects/CSC515-CTAs/CTA_Module-2/mod_images/green_channel.png`

Original color image

The same source for each comparison



Green-only color view (visual aid)

Other two channels set to 0



Result: 2D green intensity

Original green values shown in grayscale



STEP 3/5: Blue channel intensity

STEP 3/5: Blue channel intensity

The original blue values, shown as a 2D grayscale image

This result shows the blue value at each position in the original image. The green and red values are not included in this 2D array.

Low blue values appear darker, and high blue values appear brighter. A value of 0 appears black; a value of 255 appears white.

Code: `blue, green, red = cv2.split(image)`

This is the blue array returned by `cv2.split()`. In the matrix below, the 1 selects B, and the two 0s leave out the other channels. Each pixel now has one value instead of three, so the result is a 2D matrix. The grayscale display shows how much of this channel was present: 0 is none, and 255 is the highest value in this 8-bit image.

Reading the three pictures from left to right:

Original color image | Selected channel in its own color | 2D grayscale result.

The middle picture is a visual aid. It keeps the selected channel and sets the other two to zero in a separate preview array. It is not an extra processing step, and it does not convert that preview to get the grayscale result. The result on the right comes directly from `cv2.split()`.

The matrix for one pixel (@ means matrix multiplication):

$$B = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix} @ \begin{bmatrix} B \\ G \\ R \end{bmatrix}$$

$$B = 1*B + 0*G + 0*R$$

The matrix describes what happens to the color values. The code uses `cv2.split()` and `cv2.merge()` rather than multiplying each pixel by a matrix. The pixel's row and column do not change.

Example from the center pixel: `image[128, 125] = [82, 51, 78]` in BGR order.

Selected channel value: $1*82 + 0*51 + 0*78 = 82$.

Array shape: (257, 250, 3) -> (257, 250). Data type: uint8.

Saved result: `/Volumes/P-SSD/0megapy/CSU-projects/CSC515-CTAs/CTA_Module-2/mod_images/blue_channel.png`

Original color image

The same source for each comparison



Blue-only color view (visual aid)

Other two channels set to 0



Result: 2D blue intensity

Original blue values shown in grayscale



STEP 4/5: Reconstructed original

STEP 4/5: Reconstructed original

Putting the three channels back in their original positions

It put blue, green, and red back together in OpenCV's BGR order. The result has the same colors and pixel values as the decoded original image.

In the separate channel images below, lower values still appear darker and higher values appear brighter. The merge puts those values back into a color image.

Code: `reconstructed = cv2.merge((blue, green, red))`

Put each 2D channel array back into its original position: blue into B, green into G, and red into R. Splitting and merging in this order is equivalent to the identity matrix below. Every pixel keeps its original three values, so the reconstructed picture matches the decoded original.

Reading the five pictures from left to right:

Original | Blue channel input | Green channel input | Red channel input | Result.

The three middle pictures are the actual 2D arrays used by `cv2.merge()`. They are shown in output order: B, G, R. Each subtitle explains where that channel's values go in the reconstructed image.

The matrix for one pixel (@ means matrix multiplication):

```
[B']   [1  0  0]   [B]   [B]
[G'] = [0  1  0] @ [G] = [G]
[R']   [0  0  1]   [R]   [R]
```

The prime mark (') identifies an output value; it is not a derivative.

The matrix describes what happens to the color values. The code uses `cv2.split()` and `cv2.merge()` rather than multiplying each pixel by a matrix. The pixel's row and column do not change.

Example from the center pixel: `image[128, 125] = [82, 51, 78]` in BGR order.
The output pixel is `[82, 51, 78]` in BGR order.

Array shape: `(257, 250, 3) -> (257, 250, 3)`. Data type: `uint8`.

Saved result: `/Volumes/P-SSD/Omegapy/CSU-projects/CSC515-CTAs/CTA_Module-2/mod_images/reconstructed.png`

Original color image	Input: BLUE -> B slot	Input: GREEN -> G slot	Input: RED -> R slot	Result: reconstructed colors
The same source for each comparison	Original blue values become output blue	Original green values become output green	Original red values become output red	Original BGR values put back together
				

STEP 5/5: Red/green swapped (GRB)

STEP 5/5: Red/green swapped (GRB)

Using the red values for green, and the green values for red

For this merge, it keeps blue where it was and exchanges red and green. The colors change, but each pixel stays in the same row and column.

The input channel images still show lower values as darker and higher values as brighter. Their values stay the same; what changes is where it put them in the merge.

Code: `swapped = cv2.merge((blue, red, green))`

This permutation matrix rearranges the channels. Blue stays where it was. The original red values become the output's green values, and the original green values become its red values. The output still uses BGR storage, but its values come from the original channels as [B, R, G]. In RGB order, those same output colors are [G, R, B], the GRB result in the assignment.

Reading the five pictures from left to right:

Original | Blue channel input | Red channel input | Green channel input | Result.

The three input arrays keep their original values. Their titles and subtitles show where those values go: blue to blue, red to green, and green to red. The B, G, and R slots are the output's blue, green, and red positions.

The matrix for one pixel (@ means matrix multiplication):

$$\begin{bmatrix} B' \\ G' \\ R' \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} @ \begin{bmatrix} B \\ G \\ R \end{bmatrix} = \begin{bmatrix} B \\ R \\ G \end{bmatrix}$$

The prime mark (') identifies an output value; it is not a derivative.

The matrix describes what happens to the color values. The code uses `cv2.split()` and `cv2.merge()` rather than multiplying each pixel by a matrix. The pixel's row and column do not change.

Example from the center pixel: `image[128, 125] = [82, 51, 78]` in BGR order.
The output pixel is `[82, 78, 51]` in BGR order.

Array shape: (257, 250, 3) → (257, 250, 3). Data type: `uint8`.

Saved result: `/Volumes/P-SSD/Omegapy/CSU-projects/CSC515-CTAs/CTA_Module-2/mod_images/red_green_swapped.png`

Original color image	Input: BLUE → B slot	Input: RED → G slot	Input: GREEN → R slot	Result: red/green swapped
The same source for each comparison	Original blue values become output blue	Original red values become output green	Original green values become output red	Red and green exchanged; blue unchanged
				

Finished. All five results are saved, and their comparisons have been displayed.

alex.omegapy@MacStudio CSC515-CTAs %